# MacFAST – Scalable Adaptive Testing Platform
# Design and V&V Document
# January 23, 2026

## Group 10 – Capstone Project


| Full Name | Email Address (McMaster) | Student Number |
|---|---|---|
| Dai Ishida | ishidad@mcmaster.ca | 400435830 |
| Mohib Ur Rehman Siddiqui | siddim93@mcmaster.ca | 400415895 |
| Jack Fountain | fountj1@mcmaster.ca | 400333572 |
| Laeek Ahmed Shaikh | shaikl3@mcmaster.ca | 400358738 |
| Raj Rehman | moham50@mcmaster.ca | 400359821 |
| Peter Wardell | wardelp@mcmaster.ca | 400399034 |
| Jwalit Bhavinkumar Miniwala | miniwalj@mcmaster.ca | 400357388 |

# Table of Contents


Version ..... 3

Purpose ..... 4

Components ..... 4

Relationships ..... 4

Component descriptions ..... 5

Frontend ..... 5

Adaptive Testing Framework ..... 6

Backend ..... 7

Database ..... 8

UTS Servers ..... 9

Microsoft Authentication ..... 9

Details on user interface (Figma link here) ..... 10

Verification and Validation Plan ..... 12

1. Frontend ..... 12

1.1 Question Page ..... 12

1.2 Course, Unit, and Subtopic Details ..... 12

2. Backend ..... 13

2.1 Question File Parser ..... 13

2.2 Adaptive Test Framework ..... 14

2.3 User Data Compilation ..... 14

3. Authentication, and Database ..... 15

3.1 Postgres Database ..... 15

3.2 Authentication and Access Control (M365 SSO) ..... 15

4. Cross-System Validation – Pilot Phase ..... 16

Verification and Validation Results ..... 16

Appendix ..... 16

UI/ UX ..... 17

## Version

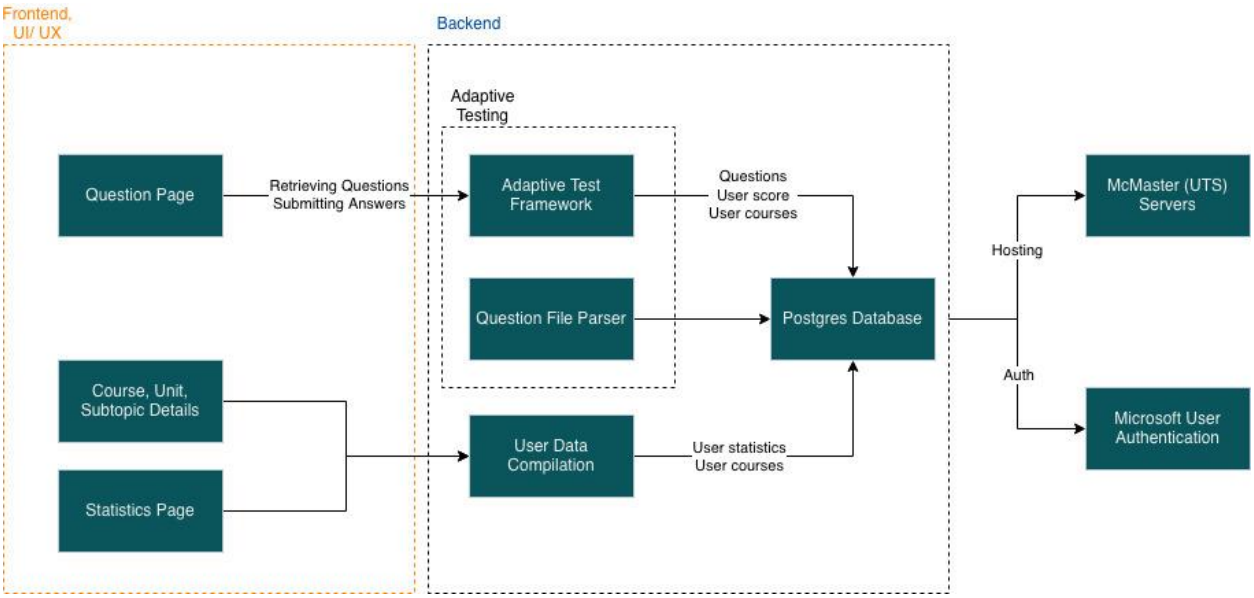
| Version | Date | Description | Authors |
|---|---|---|---|
| 0 | January 23, 2026 | Initial Draft | Everyone |

## Purpose

This document presents the initial design of the system and outlines how verification and validation will be used throughout development to ensure that the design satisfies its requirements and intended user needs. MacFAST is a web-based formative assessment study tool that uses adaptive testing techniques to adjust question difficulty based on student performance, helping them practice course material, receive feedback, and more effectively identify knowledge gaps.

# Components

*Figure: component relationships*




## Relationships

| *Requirement to meet* | *Components responsible* |
|---|---|
| Use McMaster's SSO for authentication/student login | Microsoft Authentication |
| An adaptive testing framework that will consider the student's "ability score", calculate difficulty of the question, and what topic they are currently studying to determine which question will be asked next. | Adaptive testing framework<br>Database |
| Support ~7000 total users and at least a significant fraction of concurrent users | Database<br>UTS Servers |

| | |
|---|---|
| Ability to upload test banks/ questions in multiple formats containing multiple types of questions from multiple sources and parse the questions from them<br>• CSVs from A2L quiz exports<br>• Word Documents<br>• HTML files from Scantron<br>• Crowdmark exports<br>• Ability to parse LaTeX within each of the above | Backend |
| The ability to set up classes and assign students to those classes. Students will only then access the tests for that class | Frontend<br>Backend |
| Students are only able to view their own information, ability score, and progress.<br>• Comparisons to the class average in terms of correctness, and time taken on questions<br>• Data on how much of a certain topic has been covered<br>• Data on which topics the student is struggling with | Frontend<br>Backend<br>Adaptive Testing Framework |
| Instructors can view each student's information, ability score, and progress as well as comment on questions with other instructors | Frontend<br>Backend |
| The ability to recognize "bad" questions through flags and analytics | Backend<br>Database |

# Component descriptions

## Frontend

| | |
|---|---|
| Normal behavior | The frontend provides all-user facing functionalities allowing students to login, select courses, take tests, monitor their progress, see results and statistics, and allowing instructors to upload and edit questions, create units/subtopics and update course dashboards |
| API | The frontend has several pages (paths) which allow users to interact with the platform:<br><br>`/` - Home page displays user's enrolled courses (`Course[]`) fetched via `GET /api/courses/`. Renders course cards with progress indicators.<br><br>`/auth/` - Auth0 authentication related pages<br><br>`/courses/[courseCode]/coursepage` - Course detail page with tabs (Practice Problems, Learning Objectives, Statistics, Saved Questions). Fetches |

| | |
|---|---|
| | Course object containing `units[]` and `subtopics[]` with ability scores. Displays hierarchical unit/subtopic navigation.<br><br>`/courses/[courseCode]/dashboard` - Admin-only dashboard with tabs for Questions management and Class List. Questions tab allows file upload (`FormData`) and displays question bank (`Question[]`).<br><br>`/courses/[courseCode]/[unitName]/[subtopicName]/test` - Adaptive test interface. Fetches `TestQuestion` objects via POST |
| Implementation | Built with Next.js (App Router), TypeScript, React functional components, and Shadcn UI components styled with Tailwind CSS. Authentication is handled using Auth.js |
| Undesired behavior | Unable to handle errors from the backend, poor responsiveness or layout issues on smaller screens, lacking accessibility features |

## Adaptive Testing Framework

| | |
|---|---|
| Normal behavior | The adaptive testing framework selects and sequences questions for each student based on their estimated ability and the characteristics of available questions.<br><br>Every time a student answers a question, the framework updates the student's ability estimate (using ITR - Item Response Theory) and selects the next question with appropriate difficulty. |
| Implementation | A Django app on the backend. Uses Django Views for the business logic at each of the below endpoints and serializers to convert data to and from requests.<br><br>When a user wants to begin studying a new subtopic, the frontend will get a `TestSession` from the backend and use this to determine the question to show the user. The action they take will be recorded (question was skipped, answered, and whether it was answered correctly)<br><br>The backend logic also records the question attempts in the `QuestionAttempt` table which can be compiled to provide useful info to the user. |
| Undesired behavior | Poor question selection due to incorrect IRT parameterization or bias in ability estimation resulting in students being stuck with questions that are too easy or too hard. |

### *API*

A little more in depth here because this is an important part of our app.

| POST /api/core/adaptive-test/next-question | |
|---|---|
| **Params** | subtopic_id: The public_id of the subtopic to fetch the next question from |
| **Returns** | Question \| None<br>- public_id: UUID<br>- content: string<br>- options: array |
| Retrieves a question for the user | |

| POST /api/core/adaptive-test/submit-answer | |
|---|---|
| **Params** | subtopic_id: The public_id of the subtopic to fetch the next question from<br>Option_id: the selected option |
| **Returns** | Correct_option_id: UUID<br>Explanation: string |
| Submits a user's response. Requires only subtopic_id and selected option_id, as the current question they are on is stored in the backend. Returns the id of the correct option and the explanation stored in the backend for why that option is correct. | |

| POST /api/core/adaptive-test/skip-question | |
|---|---|
| **Params** | subtopic_id: The public_id of the subtopic that the user is studying |
| **Returns** | Question \| None<br>- public_id: UUID<br>- content: string<br>options: array |

| Method | Endpoint: api/ | Description |
|---|---|---|
| `GET` | `test-session?subtopic=:subtopic_id` | Get information about the specified test session. |
| `PUT`<br>`PATCH` | `test-session?subtopic=:subtopic_id` | Create or update a test session. |
| `DELETE` | `test-session?subtopic=:subtopic_id` | Delete the test session. |

## Backend

| | |
|---|---|
| Normal behavior | The backend exposes all core business logic and data access for MacFAST. It authenticates requests (via SSO/JWT), enforces role-based authorization, manages courses, classes, users, questions, adaptive sessions, and analytics, and serves structured JSON for the frontend. |

| | |
|---|---|
| | It validates inputs, applies adaptive logic, and persists all relevant state to the database. |
| Implementation | Implemented using Django with a modular structure comprised of the following sub-components or “apps”:<br>• core: shared utilities (for the questions model and health check, other base models).<br>• courses: courses and enrolment related models and endpoints.<br>• Analytics: analytics models and data storage for `QuestionAttempt` and `UserTopicAbilityScore`<br>• sso_auth: integration with Microsoft SSO/JWT, user model and roles.<br>• MacFAST: main project configuration, URLs, settings. |
| Undesired behavior | • Race conditions under high load<br>• Failures during question parsing leading to partial or inconsistent question records.<br>• Scaling failures if load balancer for multiple instances is not setup correctly to handle heavy load |

*API*

| Method | Endpoint: api/ | Description |
|---|---|---|
| `GET, POST` | `courses/` | List all courses (Admins) or enrolled courses (Students) |
| `GET, PUT, PATCH, DELETE` | `courses/:course_code/` | View Course Details |
| `GET, POST` | `courses/:course_code/units/` | Nested: List units in this course |
| `GET, POST` | `courses/:course_code/enrolments/` | List students specifically in this course |
| `GET, POST` | `units/` | Global List. |
| `GET, PUT, PATCH, DELETE` | `units/:unit_id/` | Retrieve. Shows details + Subtopics list |
| `GET, POST` | `units/:unit_id/subtopics/` | List subtopics in this unit |
| `GET, POST` | `subtopics/` | Global List |
| `GET, PUT, PATCH, DELETE` | `subtopics/:subtopic_id/` | See method name |
| `GET, POST` | `subtopics/:subtopic_id/questions/` | List questions for this subtopic |
| `GET, POST` | `questions/` | Global List |
| `GET, PUT, PATCH, DELETE` | `questions/:question_uuid/` | Retrieve, includes Options & Images |
| `GET, POST` | `questions/:question_uuid/options/` | List options |
| `GET` | `subtopic-score/:subtopic_id` | Provides the users score as text. |
| `GET, PUT, PATCH, DELETE` | `test-session?subtopic=:subtopic_id` | Returns data about the active test session for the user. |

## Database

| | |
|---|---|
| Normal behavior | The database persistently stores all structured data for MacFAST: users, courses, classes, question banks, adaptive session state, and analytics. It allows the backend to read/write data in line with ACID principles |
| API | The backend uses the Django ORM to read and write relational data<br><br>Inputs are create/update/delete operations over models such as users, courses, classes, questions etc.<br><br>Outputs are query results (model instances, lists, and aggregates) that the backend turns into API responses. |
| Implementation | Run as a Docker container using the official PostgreSQL image. The container exposes a database port which the Django backend connects to using a username and password supplied via environment variables. |
| Undesired behavior | • Data inconsistency if migrations are not applied correctly across environments<br>• Risk of data loss if backups and recovery are misconfigured<br>• Slow queries under heavy load |

## UTS Servers

| | |
|---|---|
| Normal behavior | Hosting the production and development instances of the frontend, backend, and database.<br><br>Provides a secure, university-managed environment with network, storage, and security controls that comply with McMaster's policies. |
| API | The VMs expose application ports (e.g., HTTP/HTTPS) that will be mapped to the MacFAST domain. |
| Implementation | The servers have been set up by the UTS team where we will install and run the application containers possibly orchestrated using Kubernetes |
| Undesired behavior | • Resource exhaustion under high concurrent usage, leading to slow responses or timeouts.<br>• Misconfigured firewall or reverse proxy blocking legitimate traffic or SSO callbacks. |

## Microsoft Authentication

| | |
|---|---|
| Normal behavior | Provides Single Sign-On for MacFAST users using the university's identity provider. Users log in with their existing McMaster credentials, and no passwords are stored within MacFAST. After a successful login, the frontend and backend |

| | |
|---|---|
| | receive tokens that identify the user and their roles. |
| API | We do not have our tenant ID yet, using {tenant} as a placeholder for now.<br><br>Authorization endpoint:<br>https://login.microsoftonline.com/{tenant}/oauth2/v2.0/authorize<br>Token endpoint:<br>https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token<br>JWKS_URI:<br>https://login.microsoftonline.com/%7btenant%7d/discovery/v2.0/keys<br>Userinfo endpoint:<br>https://graph.microsoft.com/oidc/userinfo |
| Implementation | Our two confidential Microsoft Entra clients will be setup and supported by UTS to match our required configurations. |
| Undesired behavior | • Misconfigured credentials (client ID/secret or issuer URL) causing login failures across the system.<br>• Token validation errors (expired tokens, incorrect key rotation handling) causing failures.<br>• Security issues if tokens are logged, stored insecurely, or transmitted |

## Details on user interface (Figma link here)

* Our figma is becoming as outdated as we continue to progress and iterate on designs throughout development, but it is being updated to the newer look.

- Describe the UI design
  - Focused on card-based design, a UI style that is extremely common nowadays being used by companies like Instagram, Spotify, Amazon and just about every dashboard interface (Mosaic!).
  - Card-based design makes it extremely easy to look at a variety of information extremely quickly, especially if proper titles are used. A good example of this is the user's homepage where they can view all the courses that are available to them on MacFast, they only have to look across their distinct cards and the card's title to find their course quickly.
    See Fig. 1
  - For the learning objectives and practice problem sections still focused on card-based design but also incorporated visual hierarchy more intentionally where each card represents a unit that can be expanded into a list structure of subtopics.
    See Fig. 2, Fig. 3
- **user interactions** (Speaking to the professor he mentioned to focus on key actions)

| **Interaction** | **View a course page and it's subtopics** |
|---|---|
| Pre-condition | User is logged in on their homepage |
| Screenshot of pre-condition | See Fig. 1 |
| Action | User clicks on the 'details' button or course name on a course card on their homepage. |

| | |
|---|---|
| Flow | 1. Frontend redirects the user to the loading course page<br>2. Frontend sends a request with the user's auth token to the backend to `/api/courses/{course_code}/`<br>3. Backend receives request and validates the logged in user is enrolled in the course<br>4. Backend returns course data<br>5. Frontend populates course page with course data<br>6. User clicks on a unit card to be expand and view the subtopics<br>7. Frontend changes state value of accordion to open the specified unit<br>8. Accordion re-renders with correct unit open |
| Screenshot of end state | See Fig. 2, Fig. 3 |
| Exception Flow | 1. Backend receives request and validates the logged in user is enrolled in the course<br>2. User is either not logged in or not enrolled; backend returns 403 (Forbidden) |

| **Interaction** | **Start adaptive testing for a subtopic** |
|---|---|
| Pre-condition | User is logged in and on a course page and is on the "Practice Problems" tab with a unit open displaying subtopics and their practice button |
| Screenshot of pre-condition | See Fig. 2 |
| Action | User clicks on a practice button adjacent to a subtopic |
| Flow | 1. Frontend redirects user to question loading page<br>2. Frontend sends out a request to backend with user's bearer token "`/api/core/adaptive-test/next-question`"<br>3. Backend receives request, validates that the subtopic exists, the user is enrolled in the course (has permissions)<br>4. Backend returns a question with N options<br>5. Frontend populates question form with backend data |
| Screenshot of end state | See Fig. 5 |
| Exception Flow | 3. Backend receives request, validates that the subtopic exists, the user is enrolled in the course (has permissions)<br>4. User does not have permissions to the course<br>5. Backend returns 403 (Forbidden)<br>6. Frontend displays an alert informing the user they are not allowed to perform that action |

- placement of items
  - Main navigation header we followed McMaster's Brand Web Guidelines which helps to maintain a consistent experience across McMaster websites
- colour/font choices
  - We followed McMaster's Brand Web Guidelines for all our colour and font choices.

| **Colour** | **Role** |
|---|---|
| #FFFFFF | Background |

| | |
|---|---|
| #000000 | Foreground |
| #FDBF57 | Accent (used for Hover and borders) |
| #7A003C | Primary colour (Maroon) |
| #495965 (Opacity 50%) | Muted text |

McMaster branding dictates the font must be Poppins (Regular for text, semibold for buttons and bold for headings)

# Verification and Validation Plan

## 1. Frontend

Verification is done through unit tests and performance testing. Unit tests are used to ensure that UI components render correctly, handle user interactions as expected, and display appropriate states when data is loading or unavailable. Performance tests focus on confirming that page loads, navigation, and user interactions remain responsive and within acceptable latency limits during normal usage.

### 1.1 Question Page

The Question Page allows students to view practice questions, submit answers, and receive feedback during an adaptive testing session.

#### 1.1.1 Unit Tests

- Verifying that questions and multiple-choice options are rendered correctly when retrieved from backend APIs.
- Confirm that user selections are correctly registered and visually reflected in the UI.
- Verifying that answer submission triggers the appropriate API request, and that feedback is being displayed.
- Ensuring that the page handles API failures correctly, such as displaying error or loading states when data is unavailable.

#### 1.1.2 Performance Tests and Metrics

- Measuring question load time after navigation, which should remain under 1 second under normal conditions.
- Measuring response time between answer submission and feedback display, which should remain ≤ 5 seconds.
- Ensuring that the page remains responsive while waiting for backend responses.

### 1.2 Course, Unit, and Subtopic Details

This component allows students to navigate available courses, units, and subtopics before starting practice sessions.

#### 1.2.1 Unit Tests

- Verify that enrolled courses are displayed correctly for authenticated users.
- Ensuring that selecting a course loads the corresponding units and subtopics.
- Verifying correct navigation behavior between hierarchical levels.
- Checking that appropriate UI messaging is shown when no data is available.

#### 1.2.2 Performance Tests and Metrics

- Measuring initial page load time, which should remain under 2 seconds.
- Verifying that navigation between course levels (Course to Unit to Subtopic) occurs within 1 second.

### 1.3 Frontend Validation

Frontend validation will be done by pilot testing with a group of student users to evaluate usability and educational effectiveness. Students will complete tasks such as navigating courses, answering questions, and reviewing their statistics. Feedback will be collected through a questionnaire that will focus on the overall usefulness of the interface. To further validate the practical impact of the system, pilot users may also reflect on how prepared they felt before and after using the platform. High-level comparisons between platform usage and course assessment outcomes may be reviewed. This validation approach provides qualitative evidence that the frontend supports effective exam preparation and a more guided study experience.

## 2. Backend

Tests are implemented using pytest and pytest-django, with Django REST Framework's API testing utilities. Performance tests confirm that the backend meets expected performance requirements. Locust is used to simulate concurrent users and measure load behavior, while Python's time library is used for lightweight runtime measurements of specific operations.

### 2.1 Question File Parser

#### 2.1.1 Unit Tests

- Verifying that uploading a valid .docx file containing questions creates Question records with correct text content, difficulty values, and verification status.
- Verifying MCQs create Question Option records with correct option text.
- Verifying that embedded images in a .docx file generate Question Image records containing valid Base64-encoded images linked to the corresponding questions.
- Verifying that the Celery task errors when processing an unsupported file format.
- Verifying that upon successful completion of the Celery task parse_file(), all questions are stored in the DB

#### 2.1.2 Performance Tests and Metrics

- The file parser performance is evaluated by measuring the Celery parse_file() runtime from file submission to completion of question extraction.

- The average processing time should be less than 30 seconds for a 50-question file with embedded images and less than 2 minutes for files containing up to 200 questions.
- Each measurement is repeated 10 times and averaged to ensure consistency.

## 2.2 Adaptive Test Framework

### 2.2.1 Unit Tests

- Verifying that submitting an answer correctly identifies whether the response is correct or incorrect and updates the user's ability to score accordingly.
- Verifying that requesting the next question returns a question within the appropriate difficulty range based on the user's current ability score.
- Verifying that a new UserTopicAbilityScore record is created when a user starts a test for a subtopic, they have not previously attempted.
- Verifying that when no questions are available the user is prompted with actions
- Verifying that questions with higher item information values are prioritized for selection

### 2.2.2 Performance Tests and Metrics

- Adaptive testing performance is evaluated through latency, by how quickly users receive the next question and correctness feedback after submitting an answer.
- Using Locust, the next question response time should be less than 3 seconds under normal load and less than 10 seconds under peak load with up to 100 concurrent users, while the submit response time should be less than 5 seconds under normal load and less than 15 seconds under peak load.

## 2.3 User Data Compilation

### 2.3.1 Unit Tests

- Verifying that submitting an answer creates a Question Attempt record with the correct user ID, question ID, selected option, and timestamp.
- Verifying that multiple attempts can be recorded when allowed.
- Verifying that correct answers increase User Topic Ability Score per IRT logic.
- Verifying that incorrect answers decrease ability score based on parameters.
- Verifying that final ability reflects cumulative performance across attempts.

### 2.3.2 Performance tests and metrics

- The user data compilation performance is measured through the latency of recording question attempts and the computation time of IRT ability score updates, timed using time library.
- Creating a single Question Attempt record should complete within 2 seconds and under peak load simulated with Locust at 100 concurrent submissions, each record should complete within 2 seconds.
- IRT ability score calculations should be completed in real time with an average runtime of less than 1 second, measured 50 times across varied parameters to confirm consistency.

## 3. Authentication, and Database

### 3.1 Postgres Database

Verification

3.1.1 Unit and Integration Tests

- Verifying database migrations apply cleanly across environments.
- Verifying CRUD operations for core models (Users, Questions, Question Attempts, Enrolments).
- Verifying cascade deletion removes dependent records (e.g., question options, images).
- Verifying failed transactions correctly rollback without partial writes.
- Verify concurrent writes do not produce duplicate or inconsistent records.

3.1.2 Performance Tests and Metrics

- Single write latency: Creating a Question Attempt record completes within ≤ 5s under normal load.
- Concurrent write latency: Under 100 concurrent submissions, writes complete within ≤ 2 s.
- Query latency: Common read queries (fetching questions, analytics) complete within ≤ 1s.
- Connection stability: No dropped database connections during sustained Locust load tests.

3.1.3 Reliability and Recovery Tests

- Verifying database container restarts without data loss.
- Verifying persistent volumes retain data across restarts.
- Verifying backups can be restored to a clean instance.
- Verifying database recovery time ≤ 5 minutes.

Validation

- Validating analytics accuracy by comparing frontend statistics with raw database aggregates.
- Validating database stability during pilot usage with real student activity.

### 3.2 Authentication and Access Control (M365 SSO)

Verification

3.2.1 Unit Tests

- OIDC User Creation
  - Verify first login creates a Mac*FASTUser* using OIDC username claim.
  - Verify users without email fallback to *sub* as a unique identifier.
  - Verifying repeated logins does not create duplicate users.
- Access Control
  - Verifying requests without a token, expired token, or invalid token return 401 Unauthorized.
  - Verify non-admin access to admin endpoints returns 403 Forbidden.
  - Verifying role-based access restrictions are enforced consistently.

3.2.2 Performance Tests and Metrics

- Authentication performance is measured during the end-to-end login flow using Locust:
  - OIDC callback processing (excluding external provider latency) will be replicated to the user experience of McMaster's Avenue to Learn (A2L) login.
  - Session creation and role mapping: Completes within ≤ 2 seconds.
  - Under concurrent login scenarios, authentication behavior will be verified to match the stability and responsiveness observed in A2L during academic usage.

3.2.3 Security Tests

- Verifying tokens are never logged or stored persistently.
- Verifying token validation uses JWKS and handles key rotation correctly.
- Verifying all authentication traffic occurs over HTTPS.

Validation

- Validating authentication reliability with McMaster accounts in pilot deployment.
- Validating role correctness by confirming instructors and students see only authorized resources.
- Confirming authentication failures result in clear, user-facing error messages.

### 4. Cross-System Validation – Pilot Phase

- To validate MacFAST beyond technical correctness:
  - Conduct pilot testing with a small group of students.
  - Compare practice activity, adaptive progression, and perceived preparedness.
  - Where feasible, review anonymized trends between practice performance and course assessments (midterms/finals).

  This provides qualitative validation that the system infrastructure supports effective and uninterrupted learning, without making formal causal claims.

# Verification and Validation Results

The following table summarizes the results of the verification and validation activities carried out for each major component of MacFAST. These results reflect the testing performed according to the verification and validation plan and present the corresponding observations, outcomes, and supporting evidence gathered during implementation and pilot evaluation.

| Section | Scope of Testing | Expected Result | Actual Result | Testing Method / Evidence | Status |
|---|---|---|---|---|---|
| **1.1 Question Page** | Rendering of questions and answer options, answer submission flow, solution/feedback display, skip/report | The question page should load correctly, display questions and options clearly, allow users to answer or skip questions | Pilot users generally found the question page easy to use and visually clear. Positive feedback included the side-by-side solution display, skip feature, report feature, and broad range of questions. However, several issues were also observed: in | Pilot tested with 6 users; 4 post-test survey responses were collected; observational notes were recorded during and after the sessions. See Fig. 7, | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | interaction, and responsiveness of the question page during practice, and overall responsiveness of the adaptive question page. | without issue, and provide a smooth and responsive experience during adaptive testing. | some cases, the question changed on its own a few seconds after loading, the site refreshed unexpectedly, and some users reported repeated questions, or awkward follow-up prompts when selecting harder/easier question options. | Fig. 8, and Fig. 9 in Appendix for pilot session evidence and screenshots. | |
| **1.2 Course, Unit, and Subtopic Details** | Navigation through courses, units, and subtopics; visibility and clarity of the tab/selector layout; usability across device types. | Users should be able to navigate from course view to units and subtopics easily, with a clear layout and responsive interface across supported device sizes. | Pilot users were able to navigate into the platform and reach subtopics successfully. The UI was described as easy to use, visually appealing, and responsive on iPad. At the same time, some improvements were identified: the selector menu for tabs such as Practice Problems and Learning Objectives was seen as too small or inconspicuous, some text could be larger, and some buttons/features such as Skip could benefit from clearer labeling. | Observed pilot sessions with live note-taking, plus post-test survey feedback. Device responsiveness was also informally validated during iPad testing. See Fig. 7, Fig. 8, and Fig. 9 in Appendix for navigation screenshots and pilot testing images. | Partial |
| **1.4 Frontend Validation** | Overall usability and usefulness of the frontend through pilot user interaction, including navigating courses, answering questions, and reviewing progress/statistics. | Pilot users should be able to complete the main tasks successfully and should find the interface understandable and useful for guided study and exam preparation. | Frontend validation results were overall positive. Across the pilot group, users consistently commented that the UI looked good, was easy to use, and felt professional. Two users explicitly said they would be willing to use the platform for their own courses. Based on the 4 survey responses received, the average satisfaction score was 4.25/5. The main suggested improvements related to font/button prominence, clearer feature communication, more questions in the bank, and polishing unstable behaviors such as refreshing and question changes. | 6 pilot users completed testing; 4 users completed the post-test survey; observational notes were recorded during and after pilot sessions. See Fig. 7, Fig. 8, and Fig. 9 in Appendix for pilot testing images. | Pass |

| | | | | | |
|---|---|---|---|---|---|
| **2.1 Question File Parser** | Parsing of valid question files, creation of question/option/image records, handling of unsupported formats, and parser runtime. | Valid files should be parsed correctly into the database, unsupported formats should fail safely, options/images should be stored correctly, and parsing time should remain within the planned thresholds. | Unit Tests:<br>all testing criteria passed<br><br>Performance:<br>Average runtime: 197 seconds<br><br>Runtime is more than initial estimate because images were assumed to be png, but most are .emf. conversion to png increases the processing time. | Unit Tests were done using Pytest (Fig. 6), and the runtime was measured using python's time library. | Pass |
| **2.2 Adaptive Test Framework** | Correctness of answer evaluation, updating of user ability score, next-question selection logic, handling of cases where no questions are available, prioritization behavior, and endpoint latency under normal and peak load. | Correct and incorrect answers should be evaluated accurately, ability scores should update as intended, questions should be selected appropriately for the user's level, and endpoint response times should stay within the planned normal and peak-load targets. | Unit testing confirmed that the adaptive testing framework behaved as intended across the planned functional scenarios. Answer submissions were correctly classified as correct or incorrect, learner ability scores updated appropriately after each response, first-time subtopic attempts generated new UserTopicAbilityScore records, and next-question retrieval followed the expected adaptive selection logic. Edge cases involving unavailable questions were also handled correctly. In performance testing, the adaptive endpoints remained within acceptable response-time ranges under the tested conditions, showing that the framework was both functionally correct and operationally responsive. | Testing was conducted using Pytest for functional verification of the adaptive testing backend and Locust for response-time evaluation under load. Pytest covered correctness of answer evaluation, ability-score updates, next-question selection, first-attempt record creation, no-question handling, and prioritization behaviour. | Pass |

| | | | | | |
|---|---|---|---|---|---|
| **2.3 User Data Compilation** | Creation of question-attempt records, handling of repeated attempts, correctness of ability-score updates over time, cumulative performance tracking, and write/calculation timing. | User attempts should be stored accurately, repeated attempts should be handled correctly where allowed, ability-score calculations should reflect cumulative performance, and database writes/calculations should remain within the planned thresholds. | Unit testing confirmed that user data compilation behaved as intended across the planned scenarios. Submitting an answer successfully created QuestionAttempt records with the correct user, question, selected option, and recorded attempt data. Testing also confirmed that repeated attempts were stored correctly where permitted, and that ability-score updates reflected the expected cumulative behaviour over time. Correct responses increased the relevant UserTopicAbilityScore, while incorrect responses decreased it in line with the configured adaptive logic. In performance testing, record creation and ability-score calculation completed within acceptable ranges, and load testing indicated that the subsystem remained stable under concurrent submission conditions. | Verification was carried out using Pytest for backend unit testing and Locust for concurrent load testing. A total of 31 unit tests were executed, covering QuestionAttempt creation, repeated-attempt handling, cumulative ability-score updates, and correctness of score changes after both correct and incorrect responses. Performance behaviour was additionally evaluated using runtime measurements for write and calculation operations, alongside a Locust simulation of 100 concurrent users to assess stability under peak submission conditions. | Pass |
| **3.1 Postgres Database** | Database migrations, CRUD operations, cascade deletion, rollback behavior, concurrent writes, | Core database operations should function correctly across environments, concurrent operations should not create inconsistent | <u>Unit Tests:</u> All automated pytest checks for 3.1 passed. CRUD, cascades, and rollback behave as intended, and duplicate enrolments are rejected.<br><br><u>Performance Tests:</u><br>100 sequential attempt writes each stayed within the 2s limit, | Integration-style tests run with Django's test runner against the in-memory database settings used for CI. (test file: *core/tests/test_* | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | query/write latency checks, prevention of duplicate enrolments and bulk attempt consistency, and consistency of frontend statistics with raw database data. | records, performance should meet the planned latency targets, recovery should preserve data, and frontend statistics should align with underlying database records. | single writes were within the 5s limit, and common reads were within the 1s limit. ORM counts and course XP matched raw SQL. Sustained ORM queries completed without connection errors. | *postgres_database_vv.py*) Performance-marked cases use the same thresholds that we had defined in our V&V plan. (See Fig. 12 for pytest run summary) | |
| **3.2 Authentication and Access Control (M365 SSO)** | User creation on login, prevention of duplicate users, invalid/expired token handling, role-based access control, session creation timing, HTTPS/token-security checks, and validation of real login behavior during pilot/deployment testing. | Authorized users should be authenticated correctly, unauthorized or invalid access should be blocked appropriately, role restrictions should be enforced consistently, login/session behavior should meet the planned timing targets, and authentication should remain secure. | Locust testing satisfies both the speed and scale requirements of 3.2.2. Performance averaged 21.96ms and maxed out at 127ms at a scale of 100 concurrent users. | Load testing was done using mock data with mock endpoints while still completing the entire session creation and role-mapping aspect. The login functionality remained stable. See Fig 11 for Locust results | Pass |
| **4. Cross-System Validation – Pilot Phase** | End-to-end validation of the system in realistic use, including adaptive progression, stability across components, | Pilot users should be able to use the system successfully from login to practice completion, the platform should remain stable | Cross-system pilot validation was broadly positive but identified several important improvements. Across 6 pilot users, testers were able to navigate the platform and complete meaningful interactions with it. Based on the 4 completed surveys, the average satisfaction score was 4.25/5, and users | End-to-end pilot testing with 6 users, post-test survey feedback from 4 users, and observational notes collected during and after testing. See Fig. 7, Fig. 8, | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | user preparedness/perception, and general reliability during pilot activity. | during real usage, and feedback should indicate that the system supports effective study without major disruption. | generally described the interface as visually strong, smooth, and useful. At the same time, the pilot surfaced real issues affecting the end-to-end experience, including unexpected refreshing, automatic question changes, unclear adaptive progression for some users, logout persistence, and admin-page access. Overall, the pilot supports the usefulness of the system while also clearly identifying areas requiring follow-up. | and Fig. 9 in Appendix for pilot testing images and evidence. Also refer to the post-testing survey link in the Appendix. | |

# Appendix

## UI/ UX

*Fig. 1*

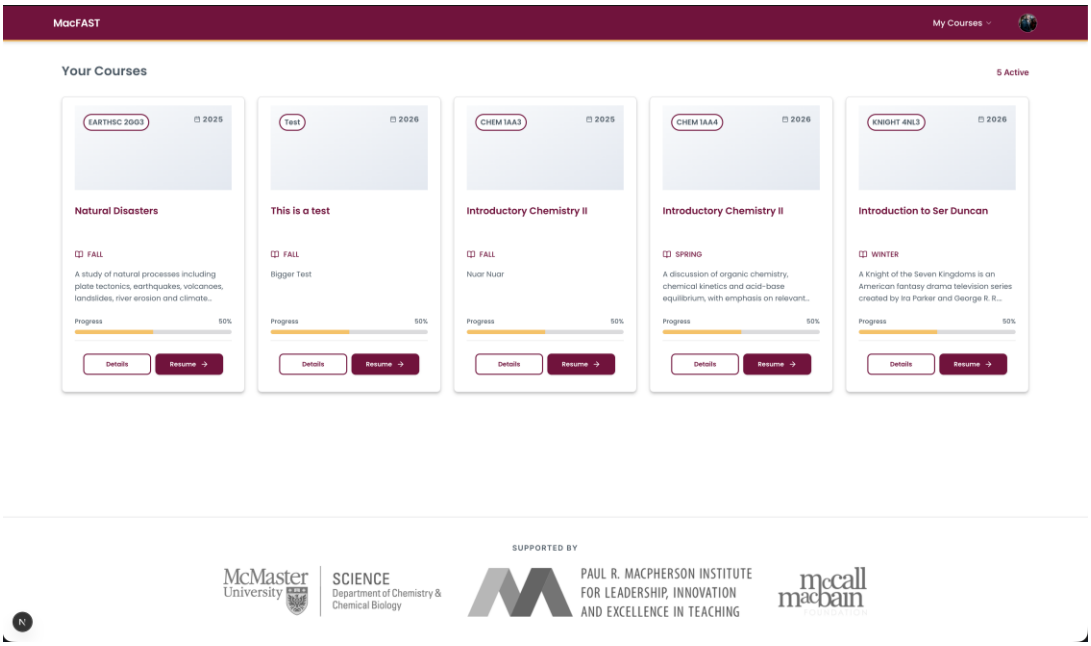



*Fig. 2*

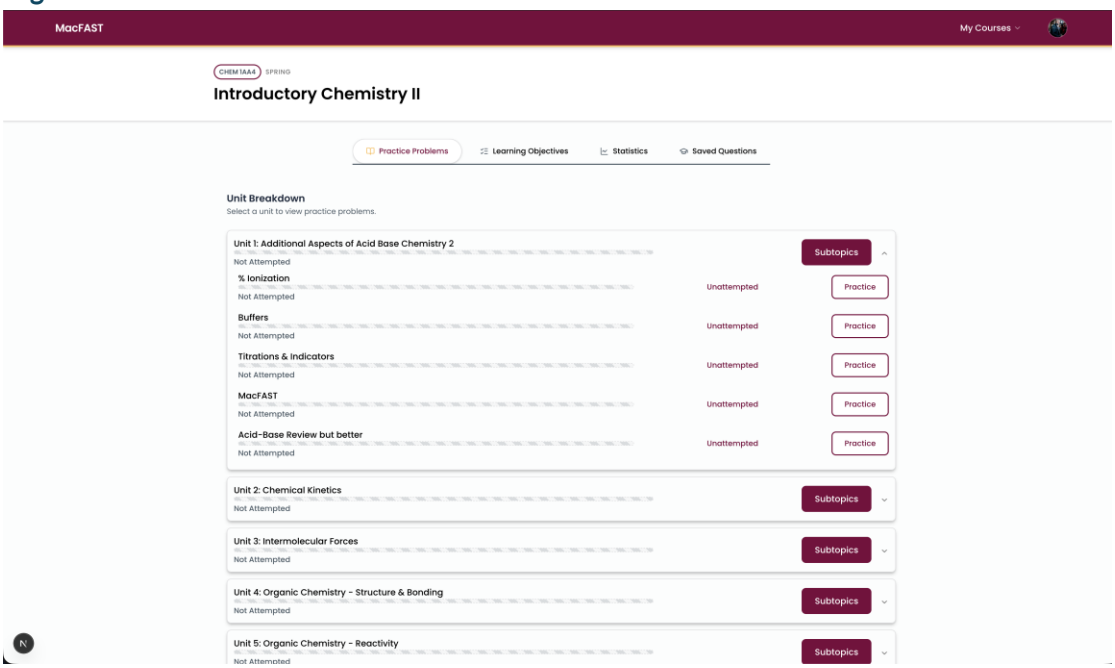


*Fig. 3*

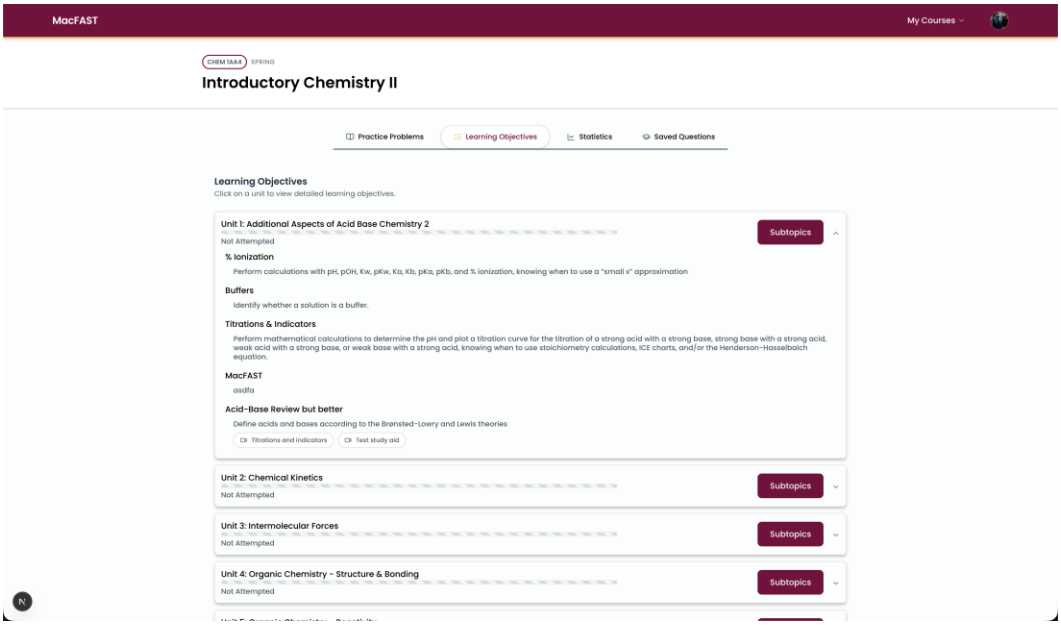

MacFAST
My Courses
CHEM 1A/A4
SPRING
Introductory Chemistry II
Practice Problems
Learning Objectives
Statistics
Saved Questions
Learning Objectives
Click on a unit to view detailed learning objectives.
Unit 1: Additional Aspects of Acid Base Chemistry 2
Not Attempted
Subtopics
% Ionization
Perform calculations with pH, pOH, Kw, pKw, Ka, Kb, pKa, pKb, and % ionization, knowing when to use a "small x" approximation
Buffers
Identify whether a solution is a buffer.
Titrations & Indicators
Perform mathematical calculations to determine the pH and plot a titration curve for the titration of a strong acid with a strong base, strong base with a strong acid, weak acid with a strong base, or weak base with a strong acid, knowing when to use stoichiometry calculations, ICE charts, and/or the Henderson-Hasselbalch equation.
MacFAST
asdfa
Acid-Base Review but better
Define acids and bases according to the Bronsted-Lowry and Lewis theories
Titrations and Indicators
Test study aid
Unit 2: Chemical Kinetics
Not Attempted
Subtopics
Unit 3: Intermolecular Forces
Not Attempted
Subtopics
Unit 4: Organic Chemistry - Structure & Bonding
Not Attempted
Subtopics

*Fig. 4*

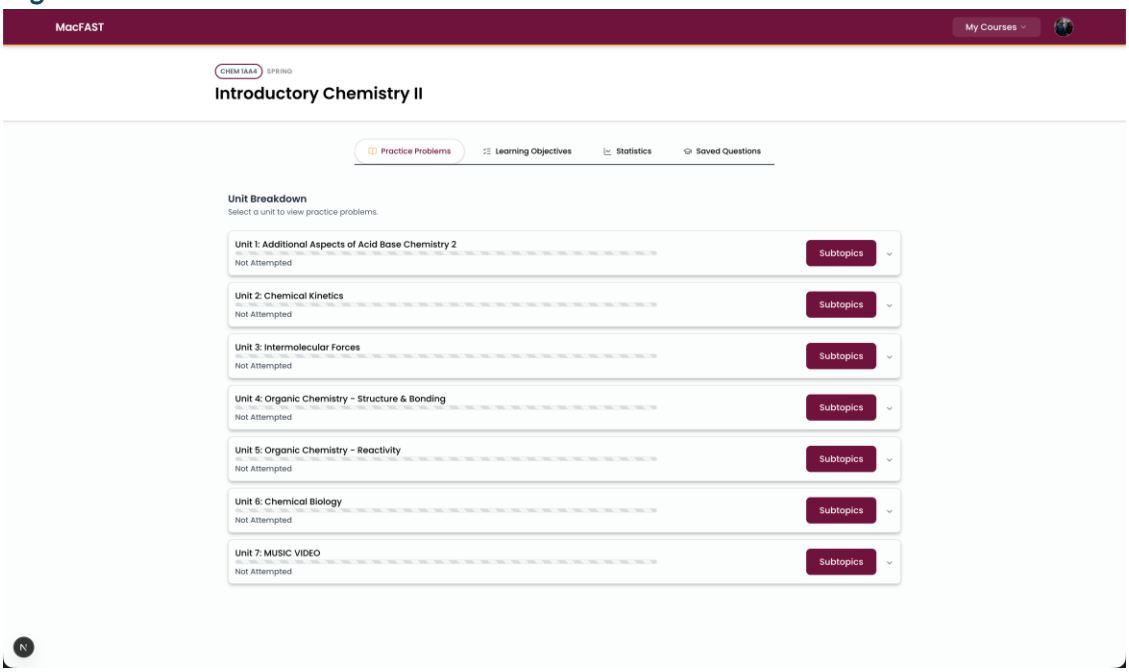

MacFAST
My Courses
CHEM 1A/A4
SPRING
Introductory Chemistry II
Practice Problems
Learning Objectives
Statistics
Saved Questions
Unit Breakdown
Select a unit to view practice problems.
Unit 1: Additional Aspects of Acid Base Chemistry 2
Not Attempted
Subtopics
Unit 2: Chemical Kinetics
Not Attempted
Subtopics
Unit 3: Intermolecular Forces
Not Attempted
Subtopics
Unit 4: Organic Chemistry - Structure & Bonding
Not Attempted
Subtopics
Unit 5: Organic Chemistry - Reactivity
Not Attempted
Subtopics
Unit 6: Chemical Biology
Not Attempted
Subtopics
Unit 7: MUSIC VIDEO
Not Attempted
Subtopics

*Fig. 5*

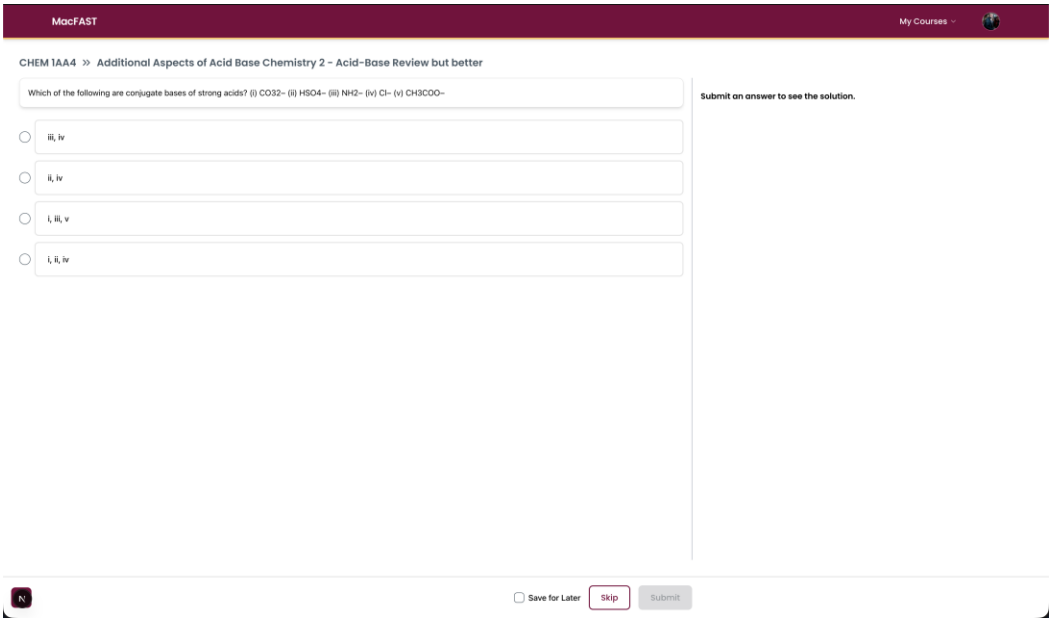



*Fig. 6*

```
collected 4 items

tests/test_parser.py::test_parser_creates_questions PASSED                    [ 25%]
tests/test_parser.py::test_parser_creates_question_options PASSED             [ 50%]
tests/test_parser.py::test_parser_creates_question_images PASSED              [ 75%]
tests/test_parser.py::test_parser_rejects_invalid_file_format PASSED          [100%]
```

*Fig. 7*

*Fig. 8*

*Fig. 9*

*Fig. 10*

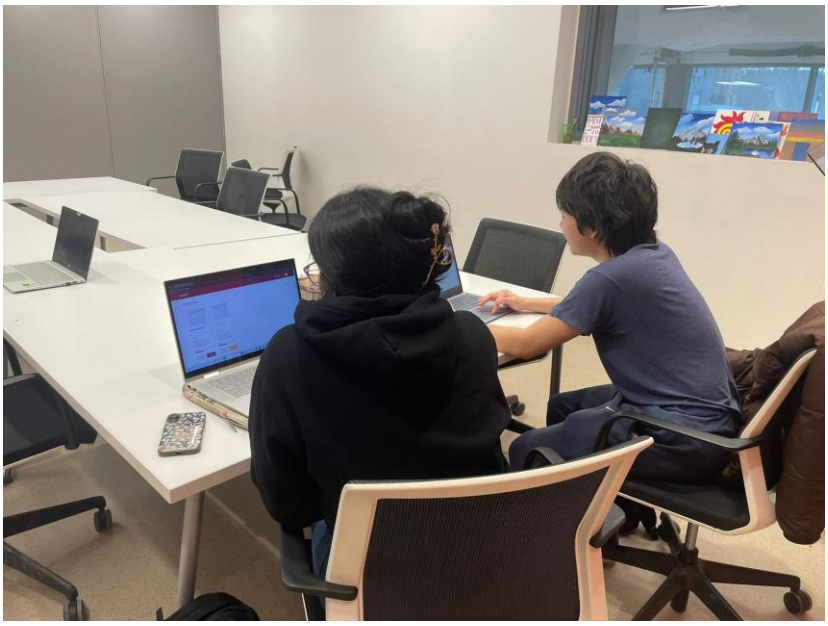


*Fig. 11*

# Locust Test Report

**During:** 4/8/2026, 9:39:48 PM - 4/8/2026, 9:40:48 PM (1 minute)
**Target Host:** http://127.0.0.1:8000
**Script:** oidc_callback_benchmark.py

## Request Statistics

| Type | Name | # Requests | # Fails | Average (ms) | Min (ms) | Max (ms) | Average size (bytes) | RPS | Failures/s |
|---|---|---|---|---|---|---|---|---|---|
| GET | 1. Init OIDC | 3312 | 1 | 13.28 | 3 | 112 | 0 | 55.66 | 0.02 |
| GET | 2. OIDC Callback Processing | 3307 | 0 | 21.96 | 15 | 127 | 858 | 55.58 | 0 |
| | Aggregated | 6619 | 1 | 17.62 | 3 | 127 | 428.68 | 111.24 | 0.02 |

## Response Time Statistics

| Method | Name | 50%ile (ms) | 60%ile (ms) | 70%ile (ms) | 80%ile (ms) | 90%ile (ms) | 95%ile (ms) | 99%ile (ms) | 100%ile (ms) |
|---|---|---|---|---|---|---|---|---|---|
| GET | 1. Init OIDC | 11 | 11 | 12 | 13 | 17 | 26 | 59 | 110 |
| GET | 2. OIDC Callback Processing | 19 | 20 | 21 | 23 | 28 | 38 | 68 | 130 |
| | Aggregated | 17 | 18 | 19 | 21 | 25 | 32 | 66 | 130 |

*Fig. 12*

```
========================================================== test session starts ==========================================================
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Faqraj\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
django: version: 6.0.4, settings: MacFAST.settings_test (from ini)
rootdir: C:\Users\Faqraj\Desktop\University Material\2025-26\CAPSTONE DEV\Capstone\backend
configfile: pytest.ini
plugins: django-4.12.0
collected 16 items

core/tests/test_postgres_database_vv.py::TestMigrationsApplyCleanly::test_no_pending_migrations_when_fully_migrated PASSED         [  6%]
core/tests/test_postgres_database_vv.py::TestMigrationsApplyCleanly::test_migrate_command_idempotent PASSED                       [ 12%]
core/tests/test_postgres_database_vv.py::TestCoreModelCrud::test_user_crud PASSED                                                 [ 18%]
core/tests/test_postgres_database_vv.py::TestCoreModelCrud::test_question_and_options_crud PASSED                                 [ 25%]
core/tests/test_postgres_database_vv.py::TestCoreModelCrud::test_question_attempt_crud PASSED                                     [ 31%]
core/tests/test_postgres_database_vv.py::TestCoreModelCrud::test_enrolment_crud PASSED                                            [ 37%]
core/tests/test_postgres_database_vv.py::TestCascadeDeletion::test_deleting_question_removes_options_and_attempts PASSED          [ 43%]
core/tests/test_postgres_database_vv.py::TestTransactionRollback::test_integrity_error_in_atomic_rolls_back_enrolment PASSED       [ 50%]
core/tests/test_postgres_database_vv.py::TestConcurrentWrites::test_enrolment_unique_constraint_prevents_duplicate_row PASSED      [ 56%]
core/tests/test_postgres_database_vv.py::TestConcurrentWrites::test_many_question_attempt_inserts_all_distinct_rows_persist PASSED [ 62%]
core/tests/test_postgres_database_vv.py::TestDatabasePerformanceThresholds::test_single_question_attempt_write_within_budget PASSED [ 68%]
core/tests/test_postgres_database_vv.py::TestDatabasePerformanceThresholds::test_many_writes_each_complete_within_budget PASSED    [ 75%]
core/tests/test_postgres_database_vv.py::TestDatabasePerformanceThresholds::test_common_read_queries_within_budget PASSED          [ 81%]
core/tests/test_postgres_database_vv.py::TestDatabasePerformanceThresholds::test_sustained_queries_no_connection_failure PASSED    [ 87%]
core/tests/test_postgres_database_vv.py::TestAnalyticsMatchRawAggregates::test_attempt_counts_match_raw_sql PASSED                [ 93%]
core/tests/test_postgres_database_vv.py::TestAnalyticsMatchRawAggregates::test_course_xp_matches_stored_row PASSED                [100%]

=================================================== 16 passed in 126.30s (0:02:06) ===================================================
```

*Post-testing Survey Link:*

Post-testing feedback survey – Fill in form

Post-testing feedback survey.xlsx